

HMC / CVS C Implementasyonu

Ana fikir

Bu dokümanın amacı, CVS_code.slx, HMC.slx, cvs_model.c, cvs_model.h, plot_output.m, run.bat ve output.csv dosyalarını profesyonel bir çalışma rehberine dönüştürmektir.

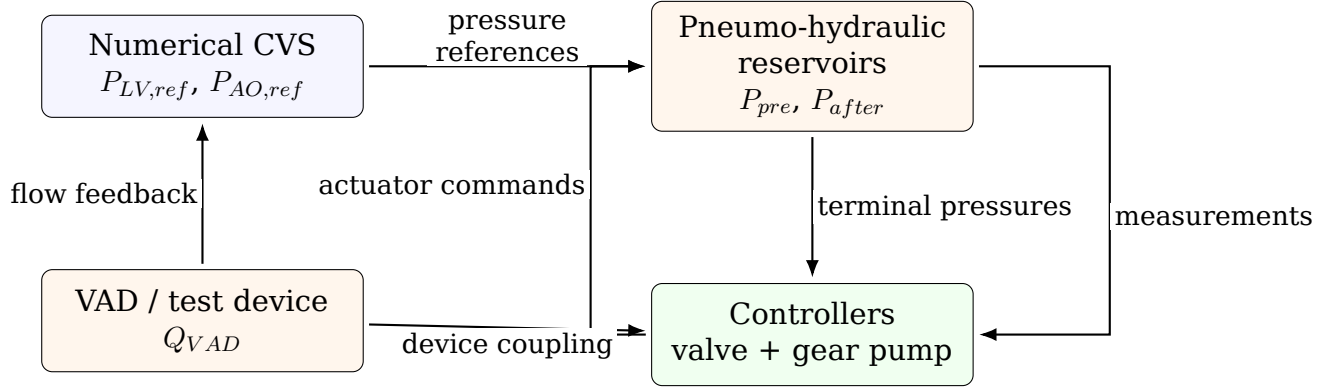
İçindekiler

1	Bağlam: HMC içinde CVS C kodunun yeri	3
2	Dosyaların rolü	4
3	CVS modelinin matematiksel temeli	5
3.1	Elektrik-hidrolik analogi	5
3.2	Pressure denklemi nereden gelir?	5
3.3	Flow denklemi nereden gelir?	6
3.4	Volume denklemi nereden gelir?	6
4	C modelinin state yapısı	7
5	Kodun çalışma sırası	8
6	Başlık dosyası: API yüzeyi	9
6.1	Chamber enum	9
6.2	CVS_Parameters	10
6.3	CVS_State	10
6.4	CVS_Simulation	10
7	Fonksiyon dokümantasyonu	11
7.1	Yardımcı inline fonksiyonlar	11
7.2	Simülasyon lifecycle fonksiyonları	11
7.3	Model hesaplama fonksiyonları	12
7.4	Output fonksiyonları	13
8	Kodda ilk düzeltilmesi gereken mühendislik noktaları	13
9	Gerekli programlar ve mini kurulum rehberi	14
9.1	Önerilen araçlar	14
9.2	MSYS2 ile GCC kurulumu	14
9.3	CMD ile run.bat çalıştırma	14
9.4	Daha açık bir derleme komutu	14

10 CVS kodu ile HMC modeli arasındaki açık geliştirme başlıkları	15
10.1 Hastalık ve senaryo koşulları C koduna nasıl eklenebilir?	15
10.2 Üretilmesi beklenen çözüm tipleri	16
11 HMC.slx içindeki diğer parçalar C’de nasıl temsil edilebilir?	16
12 Doğru çalışmayı kontrol etmek için doğrulama hiyerarşisi	17
13 STM32’ye taşıma için önerilen dosya ayrımı	18
13.1 Önerilen refactor API	18
14 Safety check’lerin koyulacağı yerler	19
15 Çalışma roadmap’i	19
16 Okuyucu için kontrol listesi	20

1 Bağlam: HMC içinde CVS C kodunun yeri

Hybrid Mock Circuit (HMC), kardiyovasküler sistemi iki ortamda birlikte temsil eder: *in silico* numerik model ve *in vitro* fiziksel sistem. Numerik CVS modeli basınç, hacim ve akış dinamiklerini üretir; fiziksel sistem ise bu referansları rezervuarlar, valfler, pompalar ve sensörler üzerinden takip eder. Bu rehberde incelenen C kodu, tam HMC'nin yalnızca **CVS numerik çekirdeğini** sadeleştirilmiş biçimde temsil eder.



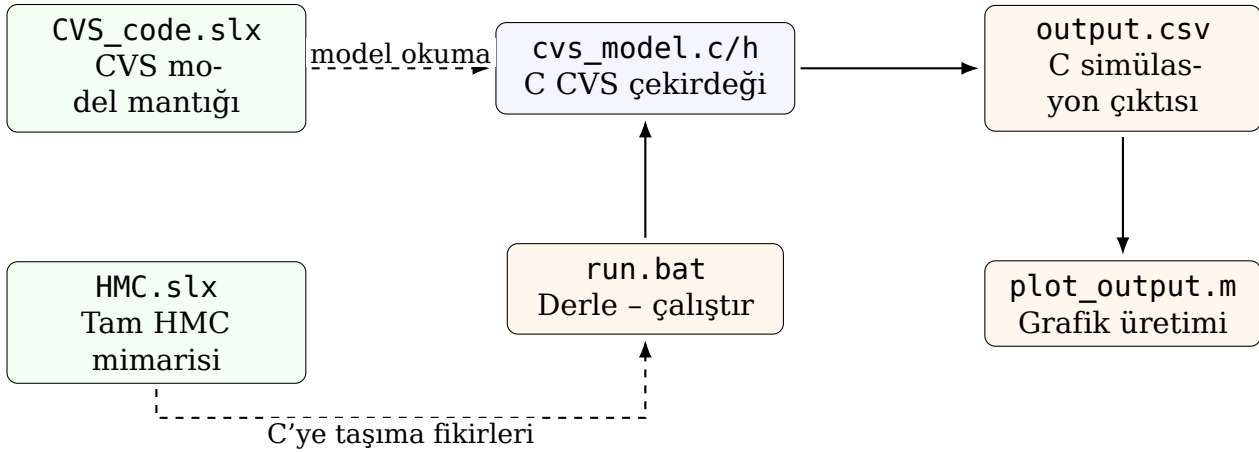
Şekil 1: Tam HMC döngüsünde numerik CVS, fiziksel sistem ve test cihazı arasındaki bilgi akışı.

Bu C demosunda fiziksel geri besleme ve kontrolcü katmanları yoktur. Kod, kapalı bir 7 durumlu CVS halkasını standalone çalıştırır. Bu nedenle dosyanın doğru okunması şu ayrımı gerektirir:

- **Bu C kodu:** CVS state'lerini, elastance, pressure, flow ve volume güncellemelerini gösterir.
- **Bu C kodunda henüz olmayanlar:** HMC'deki hastalık senaryoları, VAD flow geri beslemesi, pressure controller, gear pump controller, sensor filtering, actuator mapping ve safety supervisor.
- **Roadmap hedefi:** Önce bu sade CVS çekirdeğini temiz ve taşınabilir hale getirmek, sonra HMC'nin diğer parçalarını aynı mimariye eklemek.

2 Dosyaların rolü

Dosya	Rol
CVS_code.slx	CVS modelinin Simulink içinde, C kodu mantığına yakın kurulmuş hali. Kodun matematiksel yapısını anlamak için referans olarak incelenebilir.
HMC.slx	HMC sisteminin daha geniş modelini içerir. İçindeki CVS, pressure reference generation, fiziksel sistem, denetleyici ve interface mantığı C'ye taşınacak mimariyi düşünmek için ana kaynaktır.
cvs_model.h	Public API, enumlar, veri yapıları, fonksiyon prototipleri ve küçük yardımcı fonksiyonlar.
cvs_model.c	CVS modelinin çalıştırılabilir PC demosu. Parametreleri kurar, her adımda elastance, pressure, flow ve volume hesaplar, CSV çıktısı üretir.
plot_output.m	output.csv dosyasını okuyup pressures, volumes, flows ve LV pressure-volume loop grafiklerini çizer. Zorunlu doğrulama aracı değil, hızlı görselleştirme aracıdır.
run.bat	Windows ortamında C kodunu derlemek ve çalıştırmak için pratik başlangıç dosyası. CMD üzerinden çalıştırılabilir.
output.csv	C modelinin ürettiği ham çıktı. Debug, grafik ve temel sanity check için kullanılır.



Şekil 2: Dosyaların çalışma akışındaki ilişkisi.

3 CVS modelinin matematiksel temeli

3.1 Elektrik-hidrolik analogi

Kod, kardiyovasküler sistemi lumped-parameter bir hidrolik devre gibi ele alır. Matematiksel olarak bu sistem, elektrik devresi analogisi ile yazılır:

Elektrik devresi	Hidrolik/ CVS karşı- lığı	Yorum
Voltage V_e	Pressure P	İki nokta arasındaki fark akışı sürer.
Current I	Flow Q	Birim zamanda geçen hacim.
Charge q_e	Volume V	Bir compartment içinde depolanan miktar.
Capacitance C	Compliance C_h	Basınç değişimine karşı hacim depolama yeteneği.
Resistance R	Hydraulic resistance R_h	Akışa karşı direnç.
Conductance $1/R$	Admittance $g = 1/R_h$	Kodda flow katsayısı olarak kullanılır.
Diode	Valve	Akışın yalnızca bir yönde olmasını sağlar.

Bu analogi sayesinde üç ana denklem elde edilir:

1. Compliance/elastance ilişkisi ile pressure hesabı.
2. Ohm yasası analogisi ile flow hesabı.
3. Kütle korunumu ile volume hesabı.

3.2 Pressure denklemi nereden gelir?

Bir hidrolik compartment'ın compliance değeri, basınç değişimine karşı hacim değişimini gösterir:

$$C_h = \frac{\Delta V}{\Delta P}$$

Lineer bir compliance varsayımı yapılırsa, sıfır basınçtaki hacim V_0 referans alınır:

$$\Delta V = V - V_0$$

Dolayısıyla:

$$C_h = \frac{V - V_0}{P}$$

Buradan pressure çekilirse:

$$P = \frac{1}{C_h}(V - V_0)$$

Elastance, compliance'ın tersidir:

$$e = \frac{1}{C_h}$$

Sonuçta kodun kullandığı pressure denklemi elde edilir:

$$P_i = e_i (V_i - V_{0,i})$$

Dikkat

Kod yorumlarında bazen kısaca $P = eV$ yazılabilir; fakat gerçek C implementasyonu $P = e(V - V_0)$ şeklindedir. V_0 özellikle ventriküllerde zero-pressure volume etkisini temsil ettiği için atlanmamalıdır.

Bu denklem şu anlama gelir: Aynı volume artışı, elastance büyükse daha yüksek pressure üretir. Bu yüzden ventrikül elastance'ının sistol ve diyastol boyunca değişmesi, kalbin kasılıp gevşemesini modellemek için kritik öneme sahiptir.

3.3 Flow denklemi nereden gelir?

Elektrik devresinde Ohm yasası:

$$I = \frac{V_1 - V_2}{R}$$

Hidrolik analogide current yerine flow, voltage yerine pressure yazılır:

$$Q = \frac{P_1 - P_2}{R_h}$$

Admittance tanımı:

$$g = \frac{1}{R_h}$$

Bu nedenle iki chamber arasındaki outlet flow:

$$Q_i = g_i (P_i - P_{next(i)})$$

Kodda $g[i]$, $i \rightarrow next(i)$ yolunun admittance değeridir. Örneğin:

$$Q_{LV} = g_{LV}(P_{LV} - P_{AO})$$

Bu, LV'den AO'ya giden aortic valve akışını temsil eder.

Kalp kapakçıkları diyot gibi düşünüldüğü için bazı çıkışlarda ters flow engellenir:

$$Q_i = g_i \max(0, P_i - P_{next(i)})$$

C kodunda bu kural RA, RV, LA ve LV çıkışları için uygulanır. Bu durum tricuspid, pulmonary, mitral ve aortic valve davranışını temsil eder.

3.4 Volume denklemi nereden gelir?

Her chamber için kütle/hacim korunumu uygulanır. Bir compartment'a giren flow, çıkan flow'dan büyükse volume artar; çıkan flow büyükse volume azalır:

$$\frac{dV_i}{dt} = Q_{in,i} - Q_{out,i}$$

C kodundaki halka yapısında her chamber'ın bir önceki chamber'dan gelen tek bir inlet flow'u ve bir sonraki chamber'a giden tek bir outlet flow'u vardır:

$$Q_{in,i} = Q_{prev(i)}, \quad Q_{out,i} = Q_i$$

Bu nedenle:

$$\dot{V}_i = Q_{prev(i)} - Q_i$$

Bilgisayarda sürekli zaman türevi doğrudan işletilemediği için explicit Euler integrasyonu kullanılır. Türev tanımından:

$$\dot{V}_i(t) \approx \frac{V_i(t + \Delta t) - V_i(t)}{\Delta t}$$

Buradan:

$$V_i(t + \Delta t) = V_i(t) + \dot{V}_i(t)\Delta t$$

Kontrol noktası

Kapalı CVS halkasında dışarıdan volume eklenmiyor veya çıkarılmıyorsa toplam volume yaklaşık sabit kalmalıdır:

$$V_T = \sum_i V_i$$

Bu büyüklük, C kodu için en basit sanity check'lerden biridir.

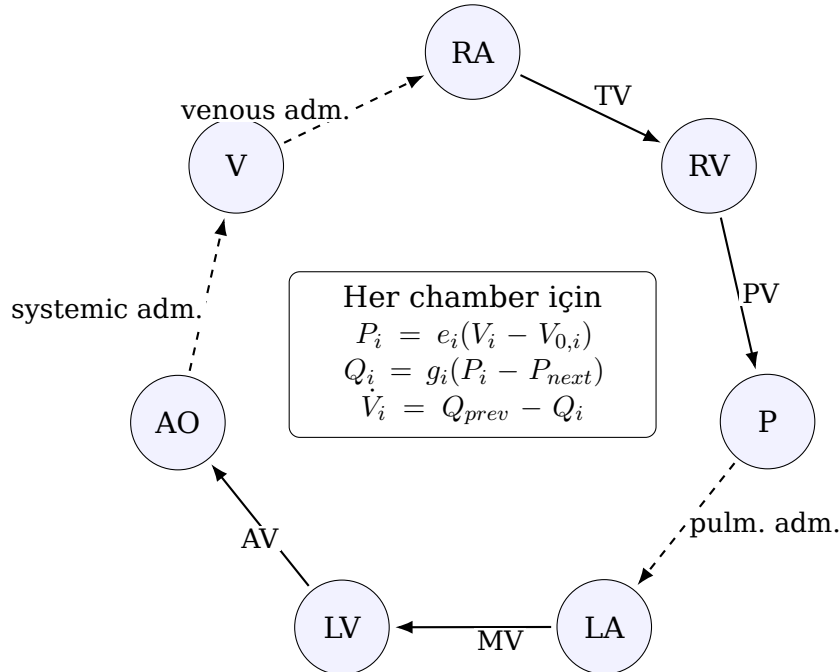
4 C modelinin state yapısı

C kodundaki state sırası şöyledir:

$$RA \rightarrow RV \rightarrow P \rightarrow LA \rightarrow LV \rightarrow AO \rightarrow V \rightarrow RA$$

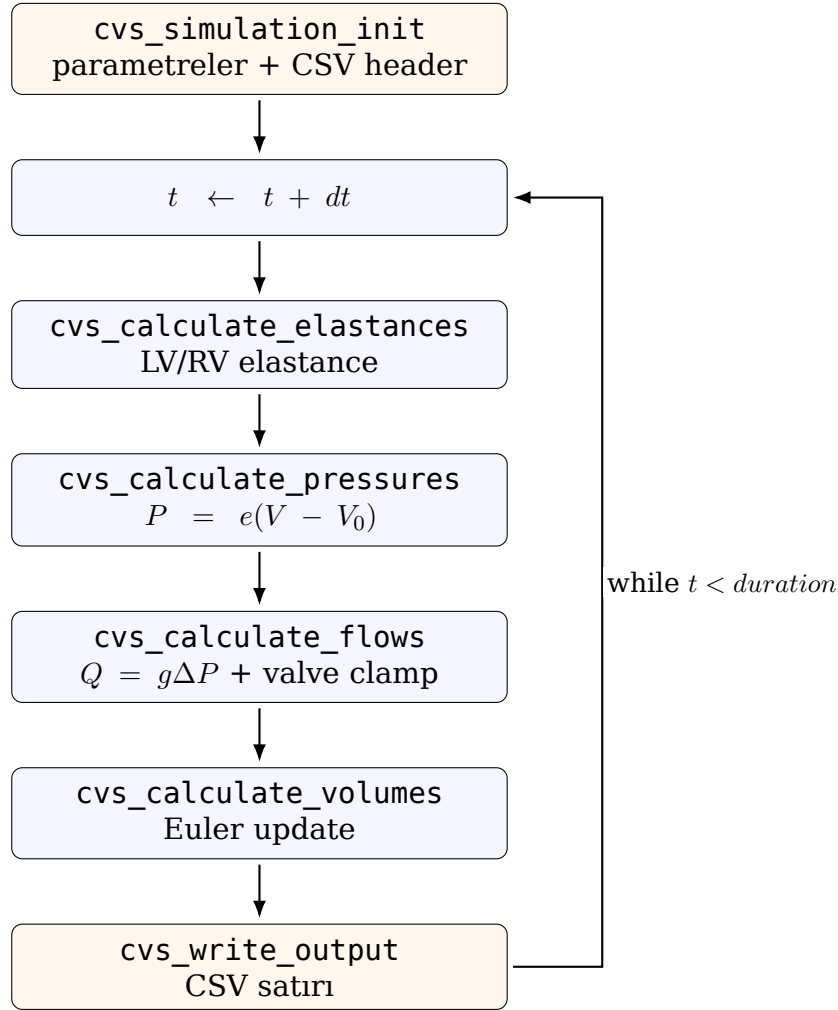
Burada P, pulmonary circulation; AO, aorta; V, veins anlamına gelir. Her chamber için üç ana büyüklük tutulur:

V_i volume, P_i pressure, Q_i outlet flow.



Şekil 3: C kodundaki 7 state'li CVS halkası.

5 Kodun çalışma sırası



Şekil 4: Bir simülasyon adımının işlem sırası.

Dikkat

Mevcut sırada pressure ve flow, volume update öncesindeki volume değerlerinden hesaplanır; CSV satırı ise volume update sonrasında yazılır. Bu eğitim demosu için kabul edilebilir olabilir, fakat embedded refactor sırasında output zamanlaması açıkça tanımlanmalıdır: *pre-step output* mu, *post-step output* mu?

6 Başlık dosyası: API yüzeyi

6.1 Chamber enum

```
typedef enum Chamber
```

Amaç: State dizilerinde index karışıklığını önlemek.

Sıra: RA, RV, P, LA, LV, AO, V.

Kritik kural: Q[i] her zaman i -> next(i) yolunu temsil eder. CHAMBER_COUNT gerçek chamber değildir; array boyutudur.

Enum değeri	Fizyolojik anlam
CHAMBER_RA	Right atrium. Çıkışı tricuspid valve gibi tek yönlü düşünülür.
CHAMBER_RV	Right ventricle. Çıkışı pulmonary valve gibi tek yönlü düşünülür.
CHAMBER_P	Pulmonary circulation. P'den LA'ya outlet admittance ile bağlıdır.
CHAMBER_LA	Left atrium. Çıkışı mitral valve gibi tek yönlü düşünülür.
CHAMBER_LV	Left ventricle. Çıkışı aortic valve gibi tek yönlü düşünülür.
CHAMBER_AO	Aorta / systemic arterial compartment.
CHAMBER_V	Veins / systemic venous compartment.

6.2 CVS_Parameters

CVS_Parameters

Model parametrelerinin tek yerde toplandığı yapıdır. MCU portunda bu yapı ikiye ayrılabilir: *constant calibration parameters* ve *runtime tunable parameters*.

Alan	Açıklama
heart_rate	Heart rate [bpm]. Beat period $T = 60/HR$ bu değerle hesaplanır. Sıfır veya negatif olamaz.
e_LV_max, e_RV_max	Maksimum LV/RV elastance değerleri. Contractility etkisini belirler.
V0[CHAMBER_COUNT]	Zero-pressure volumes. Kodda özellikle LV ve RV için 10 mL set edilir.
e[CHAMBER_COUNT]	Chamber elastance dizisi. LV/RV her adımda güncellenir.
g[CHAMBER_COUNT]	Outlet admittance dizisi. $g[i]$ yolun $i \rightarrow next(i)$ olduğunu unutmayın.
LV_*	LV elastance eğrisinin parametreleri: normalized elastance, end-diastolic pressure-volume ilişkisi ve e_{min} hesabında kullanılır.
RV_*	RV elastance aktivasyon fonksiyonu ve e_{min} hesabı için parametreler. RV_k0 ve RV_k1, HR'ye bağlı hesaplanır.

6.3 CVS_State

CVS_State

Simülasyonun anlık durumudur. Her array aynı enum sırası ile okunmalıdır.

Alan	Açıklama
V[]	Chamber volumes. Integration ile güncellenen ana state değişkenidir.
P[]	Chamber pressures. V[] ve e[] üzerinden hesaplanan algebraic output'tur.
Q[]	Outlet flows. Q[CHAMBER_LV] örneğin LV'den AO'ya flow'dur.

6.4 CVS_Simulation

CVS_Simulation

Zaman ve dosya çıktısı bilgisini tutar. Bu yapı PC demosu için uygundur; STM32 portunda FILE* alanı çıkarılmalı veya debug derlemesine özel bırakılmalıdır.

Alan	Açıklama
t	Current simulation time [s].
dt	Fixed time step [s]. Demo değeri 1 ms'dir.
duration	Toplam simülasyon süresi [s].
output_file_path	CSV dosya adı.
output_file	Açılmış dosya pointer'ı. MCU real-time loop içinde kullanılmamalıdır.

7 Fonksiyon dokümantasyonu

7.1 Yardımcı inline fonksiyonlar

cv_s_next_chamber(Chamber c)

Girdi: Chamber index.

Çıktı: Halkada bir sonraki chamber. V'den sonra tekrar RA.

Güvenlik davranışı: Geçersiz index için CHAMBER_RA döner.

Kullanım: Flow hesabında outlet node'u bulmak için kullanılır.

cv_s_prev_chamber(Chamber c)

Girdi: Chamber index.

Çıktı: Halkada bir önceki chamber. RA'dan önce V.

Kullanım: Net volume derivative için inlet flow'u bulmak.

cv_s_is_valve_outlet(Chamber c)

Çıktı: true ise outlet tek yönlü valve gibi davranır. Şu an RA, RV, LA ve LV için true döner.

Sonuç: Bu chamber'lardan çıkan flow negatif olamaz; $\max(0, \Delta P)$ uygulanır.

7.2 Simülasyon lifecycle fonksiyonları

void cv_s_simulation_init(CVS_Simulation *sim, CVS_Parameters *params, CVS_State *state)

Amaç: Simülasyonu başlatır. CSV dosyasını açar, header yazar, parametreleri set eder, başlangıç volume değerini verir.

Yan etkiler: sim->output_file açılır; params içindeki birçok alan overwrite edilir; state->V[CHAMBER_V] = 5000 yapılır.

Refactor notu: Bu fonksiyon hem dosya açıyor hem model parametresi yüklüyor. MCU portunda bu iki sorumluluk ayrılmalıdır: cv_s_load_default_params() ve logger_open() gibi.

```
void cvs_simulation_step(CVS_Simulation *sim, CVS_Parameters
*params, CVS_State *state)
```

Amaç: Bir fixed-step simülasyon adımı çalıştırır.

Sıra: $t+ = dt$; elastance; pressure; flow; volume; CSV output.

Embedded notu: STM32 portunda bu fonksiyonun dosya yazmayan, sadece state güncelleyen bir versiyonu gerekir.

```
void cvs_simulation_run(CVS_Simulation *sim, CVS_Parameters
*params, CVS_State *state)
```

Amaç: $sim \rightarrow t < sim \rightarrow duration$ olduğu sürece step çalıştırır.

Yan etkiler: Döngü bitince `cvs_simulation_close()` çağrılır.

MCU notu: Gerçek zamanda bu fonksiyon kullanılmaz. Bunun yerine timer interrupt veya RTOS periodic task her dt 'de yalnızca step fonksiyonunu çağırır.

```
void cvs_simulation_close(CVS_Simulation *sim)
```

Açık CSV dosyasını kapatır ve pointer'ı NULL yapar. PC için iyi bir alışkanlıktır; MCU portunda dosya sistemi yoksa debug logger katmanına taşınır.

7.3 Model hesaplama fonksiyonları

```
void cvs_calculate_elastances(CVS_Simulation *sim, CVS_Parameters
*params, CVS_State *state)
```

Amaç: LV ve RV elastance değerlerini o anki zamana, beat fazına ve ventrikül volume değerlerine göre günceller.

LV: $e_{min,LV}$ exponential volume ilişkisiyle; normalized elastance $e_n(t_n)$ rasyonel formülle hesaplanır.

RV: $e_{min,RV}$ exponential ilişkisiyle; systolic interval içinde $\phi(t)$ aktivasyon fonksiyonu kullanılır.

Riskler: $heart_rate \leq 0$, $dt \leq 0$, aşırı volume, expf/powf overflow, denominator sıfıra yaklaşması. Bunlar safety check gerektirir.

```
void cvs_calculate_pressures(CVS_Parameters *params, CVS_State
*state)
```

Formül: $P[i] = e[i] * (V[i] - V0[i])$.

Fiziksel anlam: Elastance büyükse aynı volume farkı daha büyük pressure üretir. Ventrikül elastance'ının zamanla değişmesi, sistol/diyastol davranışını oluşturur.

```
void cvs_calculate_flows(CVS_Parameters *params, CVS_State *state)
```

Formül: $pressure_diff = P[c] - P[next]$; $Q[c] = g[c] * pressure_diff$.

Valve kuralı: `cvs_is_valve_outlet(c)` true ise $pressure_diff = fmaxf(0, pressure_diff)$ yapılır.

Kontrol noktası: Valve flow negatif çıkmamalıdır; vessel flow yön değiştirebiliyorsa bunun model tercihi uygunluğu ayrıca yorumlanmalıdır.

```
void cvs_calculate_volumes(CVS_Simulation *sim, CVS_State *state)
```

Formül: $V_{dot}[c] = Q[prev] - Q[c]; V[c] += V_{dot}[c] * dt.$

İntegrasyon: Explicit Euler. Hızlıdır ama büyük dt ile hatayı büyütebilir.

Invariant: Kapalı halkada dış input yoksa toplam volume yaklaşık sabit kalmalıdır.

7.4 Output fonksiyonları

```
void cvs_write_output_header(CVS_Simulation *sim)
```

CSV başlığını yazar: t, sonra bütün V_* , sonra P_* , sonra Q_* kolonları. Bu sıralama plot_output.m ile uyumludur.

```
void cvs_write_output(CVS_Simulation *sim, CVS_State *state)
```

Bir zaman örneğini CSV satırı olarak yazar. PC debug için uygundur. MCU'da bu fonksiyon real-time path'ten çıkarılmalı; gerekirse DMA UART, SWO, SD card logging veya ring buffer ile ayrı düşük öncelikli task'a taşınmalıdır.

8 Kodda ilk düzeltilmesi gereken mühendislik noktaları

Dikkat

1. bool için header eksik olabilir. cvs_model.h içinde bool kullanılıyor; taşınabilir C için #include <stdbool.h> eklenmelidir.

Dikkat

2. Matematik kütüphanesi linklenmeli. Linux/GCC ortamında fmodf, expf, powf, sinf, fmaxf için çoğu zaman -lm gerekir. Windows/MSYS2 ortamında da build komutu açıkça test edilmelidir.

Dikkat

3. FILE* ve fprintf real-time kod değildir. PC demosunda eğitim amaçlıdır. STM32 üzerinde ana control loop'a sokulursa jitter üretir ve gerçek zamanlılığı bozar.

Dikkat

4. Float kullanımı bilinçli yönetilmeli. MCU hedefinde float kullanmak makul olabilir; ancak tüm sabitler, matematik fonksiyonları ve ara değişkenler gereksiz double promotion üretmeyecek şekilde incelenmelidir.

9 Gerekli programlar ve mini kurulum rehberi

Bu bölüm Windows üzerinde `run.bat` dosyasını CMD ile çalıştırmak için en basit yolu anlatır. Amaç, hızlıca gcc komutunu çalışır hale getirmektir.

9.1 Önerilen araçlar

Araç	Neden gerekli?	Resmi bağlantı
VS Code	Kod okumak, düzenlemek ve terminal açmak için pratik editör. Zorunlu değil ama önerilir.	https://code.visualstudio.com/download
MSYS2	Windows'a GCC toolchain kurmak için pratik yöntem.	https://www.msys2.org/
CMD	<code>run.bat</code> dosyasını çift tıklamak yerine kontrollü çalıştırmak için Windows Command Prompt.	Windows ile gelir.

9.2 MSYS2 ile GCC kurulumu

1. MSYS2'yi resmi sayfasından kurun: <https://www.msys2.org/>
2. Başlat menüsünden **MSYS2 UCRT64** terminalini açın.
3. Paket veritabanını güncelleyin:

```
pacman -Syu
```

Güncelleme sonrası terminal kapanırsa yeniden **MSYS2 UCRT64** açın ve devam edin:

```
pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-cmake make
```

GCC'nin kurulduğunu kontrol edin:

```
gcc --version
```

9.3 CMD ile `run.bat` çalıştırma

1. Proje klasörünün içinde boş alana tıklayın.
2. Adres çubuğuna `cmd` yazıp Enter'a basın. CMD doğrudan proje klasöründe açılır.
3. Şunu çalıştırın:

```
run.bat
```

GCC bulunamazsa `run.bat` içindeki PATH satırı MSYS2 kurulumuna göre düzeltilmelidir. UCRT64 kurulumu için genellikle şu daha güvenli olur:

Listing 1: Önerilen PATH satırı

```
set "PATH=C:\msys64\ucrt64\bin;C:\msys64\usr\bin;%PATH%"
```

9.4 Daha açık bir derleme komutu

Listing 2: Tek satırlık GCC derleme örneği

```
gcc -std=c11 -Wall -Wextra -Werror -Wdouble-promotion -o cvs_model.exe cvs_model.c -lm
```

Windows/MSYS2’de -lm gerekmesi bile komutta kalması taşınabilirlik açısından faydalıdır. Derleme başarıyla bittikten sonra:

```
cvs_model.exe
```

Beklenen temel çıktı, klasörde output.csv dosyasının oluşmasıdır.

10 CVS kodu ile HMC modeli arasındaki açık geliştirme başlıkları

Geliştirme ve araştırma başlığı

Mevcut C demosu, HMC içindeki CVS ile bazı noktalarda farklı sonuçlar verebilir. Ayrıca HMC modelinde bulunan hastalık koşulları ve çalışma senaryoları bu C kodunda henüz yoktur. Çalışma ekibinden beklenen şey, bu farkları yalnızca hata olarak görmek değil; hangi model varsayımından, parametre setinden, integration sırasından veya eksik scenario katmanından kaynaklandığını açıklayabilecek çözüm önerileri üretmektir.

10.1 Hastalık ve senaryo koşulları C koduna nasıl eklenebilir?

Önerilen yaklaşım, disease condition’ları doğrudan hesaplama fonksiyonlarının içine dağıtmamak; ayrı bir scenario veya condition katmanı kurmaktır.

Listing 3: Scenario katmanı için kavramsal API taslağı

```
typedef enum {
    CVS_SCENARIO_HEALTHY_REST,
    CVS_SCENARIO_HEALTHY_EXERCISE,
    CVS_SCENARIO_HFREF_REST,
    CVS_SCENARIO_HFREF_EXERCISE,
    CVS_SCENARIO_CUSTOM
} CVS_Scenario;

bool cvs_apply_scenario(CVS_Parameters *p, CVS_Scenario scenario);
```

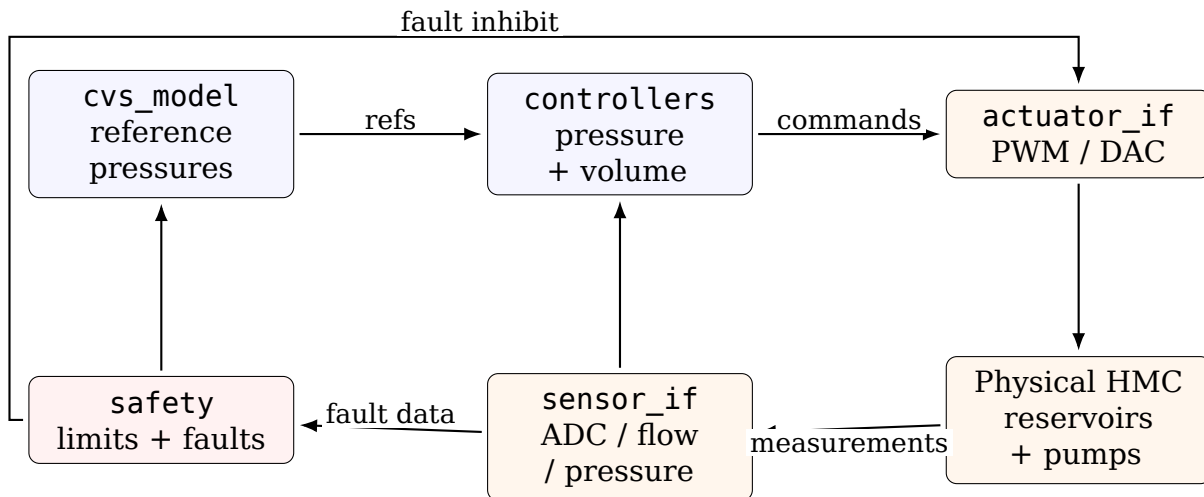
Bu yapı ile HFrEF, exercise veya assisted circulation gibi koşullar; elastance, resistance/admittance, compliance ve input flow parametrelerini kontrollü biçimde değiştirebilir. Böylece kodun ana denklemleri temiz kalır, klinik/fizyolojik senaryo yönetimi ayrı bir dosyada tutulur.

10.2 Üretilmesi beklenen çözüm tipleri

Eksik veya farklı nokta	Muhtemel C çözümü	Kontrol fikri
Hastalık parametreleri yok	Scenario preset tablosu; HFrEF için düşük $e_{LV,max}$, farklı vascular admittance/compliance değerleri	Her preset için pressure, volume ve flow aralıkları fizyolojik olarak makul mü?
Exercise koşulları yok	HR, skeletal muscle pump, vascular resistance/admittance değişimleri için runtime parameter update	Toplam volume korunuyor mu, CO beklenen yönde artıyor mu?
VAD geri beslemesi yok	Q_{VAD} input olarak LV ve AO volume denklemlerine eklenir/çıkarılır	$Q_{VAD} = 0$ iken eski modelle aynı davranıyor mu?
HMC ile farklı transient	Aynı initial condition, aynı dt , aynı integration sırası, aynı output sampling tanımlanır	İlk saniyeler ile steady-state ayrı yorumlanır.
Valve/vessel yön farkları	Valve için clamp; vessel için gerekirse model kararına göre clamp veya bidirectional flow	Negatif flow'lar bilinçli mi, numerical artifact mi?

11 HMC.slx içindeki diğer parçalar C'de nasıl temsil edilebilir?

HMC'yi STM32'ye taşıırken yalnızca CVS denklemleri yetmez. Simulink modelindeki her işlevin C tarafında bir modül karşılığı olmalıdır.



Şekil 5: STM32 portu için önerilen modüler mimari.

HMC/Simulink parçası	C karşılığı	Doğru çalıştığını kontrol etme fikri
CVS numerical model	cvs_model.c/h; fixed-step state update	Unit test: $P = e(V - V_0)$, $Q = g\Delta P$, volume conservation.
Reference pressure generation	CVS_Output struct içinde $P_{LV,ref}$, $P_{AO,ref}$	Reference sinyaller finite, bounded ve fizyolojik aralıkta mı?
Lyapunov pressure controller	pressure_controller.c/h; valve admittance veya command üretimi	Sabit referansta hata azalıyor mu? Command saturation düzgün mü?
Gear pump / volume controller	volume_controller.c/h; gear pump PWM veya speed reference	$\Delta V_h \rightarrow 0$ eğilimi var mı? Disturbance sonrası toparlıyor mu?
Sensor filtering	filter.c/h; Kalman, IIR veya moving average	Bilinen noisy sinyalde delay ve noise attenuation ölçülür.
Sensor interface	sensor_if.c/h; ADC to physical unit conversion	Kalibrasyon eğrisi ile 0-10 V giriş doğru mmHg/L/min değerine gidiyor mu?
Actuator interface	actuator_if.c/h; PWM/DAC conversion	Command sınırları, dead-zone ve offset ölçülür.
Scenario manager	scenario.c/h; healthy, HF-rEF, exercise vb.	Scenario değişince yalnızca ilgili parametreler değişiyor mu?
Safety supervisor	safety.c/h; limits, NaN/Inf, watchdog, fault state	Hatalı input enjekte edilince sistem güvenli moda geçiyor mu?
Logger	logger.c/h; UART/USB/SD veya ring buffer	Logging real-time loop'u yavaşlatıyor mu?

12 Doğru çalışmayı kontrol etmek için doğrulama hiyerarşisi

Bu rehberde ayrıca MATLAB/Simulink doğrulama akışı verilmemiştir; çünkü mevcut CVS C çıktısı daha önce kontrol edilmiştir. Bundan sonraki amaç, C mimarisinin genişletilebilir ve test edilebilir hale getirilmesidir. Bunun için doğrulama şu katmanlarda düşünülmelidir:

1. **Equation-level unit test:** Tek tek pressure, flow, volume fonksiyonlarına bilinen input verilir, beklenen output hesaplanır.
2. **Invariant test:** Kapalı CVS halkasında toplam volume korunuyor mu? Pressure/flow/volume değerlerinde NaN/Inf var mı?
3. **Scenario test:** Healthy, HF-rEF, exercise gibi koşullar parametreleri doğru değiştiriyor mu?
4. **Controller dry-run:** Fiziksel donanım bağlanmadan controller'a yapay sensor feedback verilerek command üretimi izlenir.
5. **Sensor/actuator calibration test:** ADC, DAC, PWM dönüşümleri fiziksel ölçüm cihazlarıyla doğrulanır.
6. **Hardware-in-the-loop smoke test:** Düşük basınç, düşük flow ve güvenli limitlerle tüm döngü kısa süre çalıştırılır.

Kontrol noktası

Bir HMC parçası C'ye taşındığında “çalışıyor” demek için yalnızca grafiğin benzermesi yeterli değildir. En az bir unit test, bir sınır testi, bir fault testi ve bir real-time timing ölçümü yapılmalıdır.

13 STM32'ye taşıma için önerilen dosya ayrımı

Modül	Sorumluluk
cvs_model.c/h	Sadece numerik CVS step'i. Dosya I/O yok. PC ve MCU aynı çekirdeği kullanabilmeli.
scenario.c/h	Healthy, HFrEF, exercise, custom parameter set gibi koşullar.
hmc_controllers.c/h	Lyapunov pressure controller ve backstepping gear pump controller.
sensor_if.c/h	ADC, pressure sensor, flow sensor okuma ve kalibrasyon.
actuator_if.c/h	PWM, DAC, valve driver ve motor driver çıkışları.
safety.c/h	Limit checks, NaN/Inf check, pressure/volume/flow bounds, emergency stop, saturation.
logger.c/h	Real-time loop dışında debug ve veri kaydı.
app_main.c	Timer/RTOS scheduling ve modülleri bağlama.

13.1 Önerilen refactor API

Listing 4: PC ve STM32 için ortak çekirdek API taslağı

```
typedef struct {
    CVS_Parameters params;
    CVS_State state;
    float t;
    float dt;
} CVS_Context;

typedef struct {
    float P_ref_LV;
    float P_ref_A0;
    float Q_LV_A0;
    float V_total;
} CVS_Output;

bool cvs_init_default(CVS_Context *ctx, float heart_rate, float dt);
bool cvs_apply_scenario(CVS_Context *ctx, CVS_Scenario scenario);
bool cvs_step(CVS_Context *ctx, CVS_Output *out);
bool cvs_validate_state(const CVS_Context *ctx);
```

Bu taslakta dosya yazma yoktur, main() yoktur ve her step bir output struct döndürür. Böylece aynı çekirdek PC testlerinde, HMC C prototipinde ve STM32 firmware içinde kullanılabilir.

14 Safety check'lerin koyulacağı yerler

Yer	Kontrol	Aksiyon
Init sonrası	heart_rate > 0, dt > 0, parametreler finite mi?	Hata ise init fail; real-time loop başlatma.
Elastance hesabı sonrası	e_LV, e_RV finite ve pozitif mi?	Clamp veya fault.
Pressure hesabı sonrası	Pressure aralıkları fizyolojik/sistem limitlerinde mi?	Warning, saturation veya fault.
Flow hesabı sonrası	Valve flow negatif mi? Vessel flow beklenmedik yönde mi?	Logla; gerekirse model kurallını düzelt.
Volume update sonrası	Volume negatif mi? Toplam volume drift yaptı mı?	Step'i reddet, clamp, fault veya safe state.
Actuator çıkışı öncesi	PWM/DAC command limitte mi? Basınç/akış command güvenli mi?	Saturation + fault counter.
Her real-time tick	Runtime < dt mi? NaN/Inf var mı? Watchdog besleniyor mu?	Deadline miss ise fault state.

15 Çalışma roadmap'i

Aşama	Odak	Başarı koşulu
1	C kodunu temiz derlemek	CMD üzerinden run.bat veya açık GCC komutu warning'siz çalışır, output.csv oluşur.
2	Matematiksel çekirdeği anlamak	Pressure, flow ve volume denklemlerinin nereden geldiği sözlü ve yazılı açıklanabilir.
3	API refactor tasarımı	Dosya I/O, model step ve parameter loading ayrıştırılır. main() çekirdek modelden ayrılır.
4	Scenario katmanı	Healthy/HFrEF/exercise gibi koşullar hesap fonksiyonlarına dağılımdan parametre preset'i olarak uygulanır.
5	HMC parçalarının C karşılıkları	Controller, sensor, actuator, safety ve logger modülleri için dosya ve API taslağı çıkarılır.
6	Test altyapısı	Unit test, invariant test, fault injection ve timing measurement için minimum test planı yazılır.
7	STM32 entegrasyon hazırlığı	Timer tabanlı fixed-step loop, ADC/PWM/DAC mapping ve safety supervisor iskeleti hazırlanır.

16 Okuyucu için kontrol listesi

Bu kod tabanı üzerinde çalışmaya başlayan bir kişi şu soruları cevaplayabilmelidir:

- $Q[\text{CHAMBER_LV}]$ hangi iki node arasındaki flow'u temsil eder?
- Valve outlet ile vessel outlet farkı kodda nerede uygulanır?
- $P = e(V - V_0)$ denklemi hangi fiziksel varsayımdan gelir?
- Flow denklemi Ohm yasası analogisinden nasıl türetilir?
- Volume update neden $Q_{prev} - Q_i$ şeklindedir?
- Explicit Euler integrasyonunun avantajı ve riski nedir?
- LV elastance neden her adımda değişir?
- HFrEF veya exercise koşulları C koduna nerede eklenmelidir?
- STM32'de `fprintf` neden real-time loop'a girmemelidir?
- HMC'deki controller, sensor, actuator ve safety parçalarının C karşılıkları nasıl ayrılmalıdır?